

Experiment No. 8

Aim:

To design and train a Convolutional Neural Network (CNN) on image datasets.

Theory:

Dataset Link: <https://www.cs.toronto.edu/~kriz/cifar.html>

Dataset Description:

The CIFAR-10 dataset is a widely used benchmark for image classification and object recognition.

- **Size:** 60,000 color images.
- **Splits:** 50,000 images for training and 10,000 images for testing.
- **Features:** Each image is 32x32 pixels with 3 color channels (RGB).
- **Target Variable:** A single integer ranging from 0 to 9, representing one of 10 mutually exclusive classes (Airplanes, Cars, Birds, Cats, Deer, Dogs, Frogs, Horses, Ships, and Trucks).
- **Characteristics:** The dataset is perfectly balanced, meaning there are exactly 6,000 images per class.

Mathematical Terms / Formula:

A Convolutional Neural Network processes images through specialized layers. The core mathematical operations include:

- Convolution: The convolutional layer extracts features by sliding a kernel (filter) over the input image.
- Activation Function: To introduce non-linearity into the network, the Rectified Linear Unit (ReLU) function is applied element-wise to the output of the convolution:

$$f(x) = \max(0, x)$$

- Max Pooling: Pooling reduces the spatial dimensions of the feature map, reducing computational load and providing slight translation invariance.

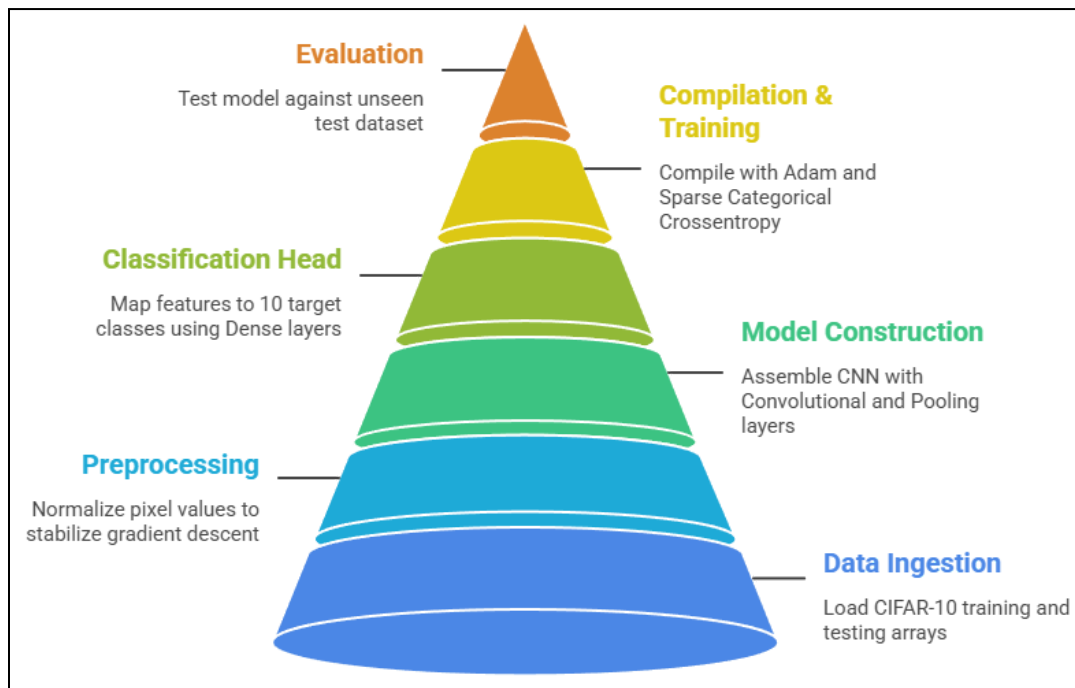
Algorithm Limitations:

While CNNs are powerful for spatial data, they have distinct limitations:

- **Computational Expense:** CNNs require significant memory and processing power (often necessitating GPUs) to train effectively, especially as the number of layers and image resolution increases.
- **Lack of Spatial Invariance:** Standard CNNs are not inherently invariant to large rotations or significant scaling of the object within the image. Data augmentation is usually required to compensate for this.
- **Data Hunger:** CNNs are prone to overfitting if trained on small datasets. They generally require massive amounts of labeled data to generalize well from scratch.
- **Vulnerability to Adversarial Attacks:** CNNs can be easily fooled by adding subtle, human-imperceptible noise to an image, causing the network to misclassify it entirely.

Workflow:

- **Data Ingestion:** Load the CIFAR-10 training and testing arrays.
- **Preprocessing:** Normalize the integer pixel values (0-255) to floating-point values (0.0-1.0) to stabilize gradient descent and speed up convergence.
- **Model Construction:** Assemble the CNN using alternating Convolutional layers (for feature extraction) and Max Pooling layers (for spatial downsampling). Flatten the final 2D feature maps into a 1D vector.
- **Classification Head:** Pass the flattened vector through Dense (fully connected) layers to map the extracted features to the 10 target classes.
- **Compilation & Training:** Compile the model using the Adam optimizer and Sparse Categorical Crossentropy loss. Train the model using forward and backward propagation over a set number of epochs.
- **Evaluation:** Test the trained model against the unseen test dataset to evaluate its generalization.



Performance Analysis:

When analyzing the model's output, rely on the following metrics:

- **Accuracy:** The percentage of correctly classified images in the test set. For a basic CNN on CIFAR-10, an accuracy of 65% to 75% is standard.
- **Loss:** The value of the cross-entropy function. A decreasing training loss with an increasing validation loss indicates overfitting.
- **Confusion Matrix:** Used to identify specific classes the model struggles with (e.g., confusing "Cats" with "Dogs" due to physical similarities).

Hyperparameter Tuning:

Hyperparameter	Tuning Strategy	Impact on Performance
Learning Rate	Tested values like 0.001, 0.0005, and 0.01.	Too high causes erratic convergence; too low causes slow training. Optimizing it ensures the model reaches the global minimum efficiently.
Batch Size	Compared sizes of 32, 64, and 128.	Smaller batches offer a regularizing effect but take longer to train. Larger batches process faster but might converge to sharp, less generalizable minima.
Dropout Rate	Inserted a Dropout layer (0.2 to 0.5) before the final Dense layer.	Randomly disables neurons during training. It heavily reduces overfitting, significantly improving validation accuracy on unseen data.
Number of Filters	Scaled from 32-64-64 to 64-128-128.	More filters capture more complex patterns but increase computational cost and the risk of overfitting.

Code:

```
import tensorflow as tf
from tensorflow.keras import layers,
models
import matplotlib.pyplot as plt

# 1. Load the CIFAR-10 Dataset
(train_images, train_labels),
(test_images, test_labels) =
tf.keras.datasets.cifar10.load_data()

# 2. Data Preprocessing: Normalize
pixel values to be between 0 and 1
train_images, test_images =
train_images / 255.0, test_images /
255.0

# 3. Define the CNN Architecture
model = models.Sequential([
    layers.Conv2D(32, (3, 3),
activation='relu', input_shape=(32, 32,
3)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3),
activation='relu'),
```

```
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3),
activation='relu'),
    layers.Flatten(),
    layers.Dense(64,
activation='relu'),
    layers.Dropout(0.5), # Added for
regularization during hyperparameter
tuning
    layers.Dense(10)
])

# 4. Compile the Model
model.compile(optimizer='adam',

loss=tf.keras.losses.SparseCategoricalC
rossentropy(from_logits=True),
metrics=['accuracy'])

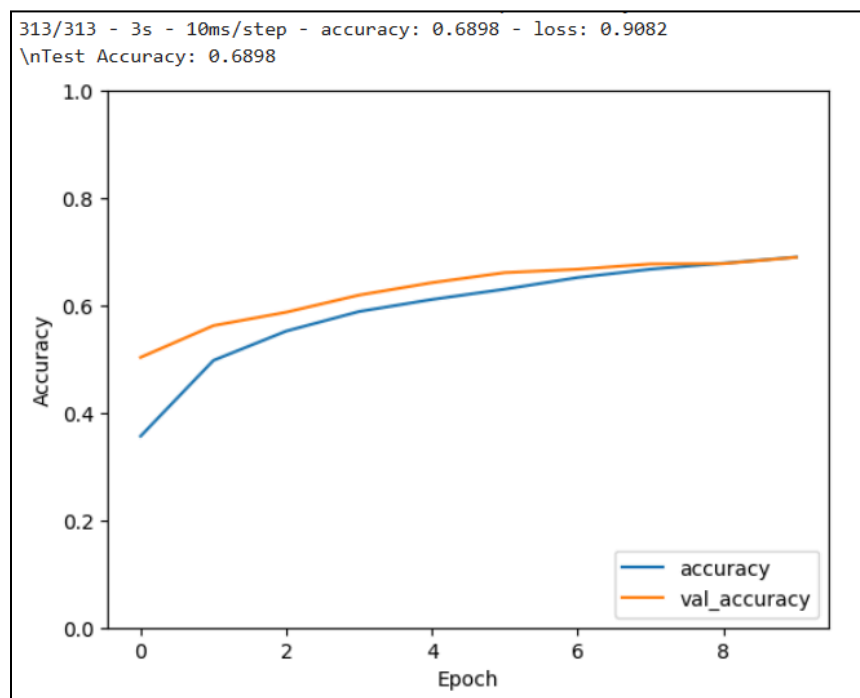
# 5. Train the Model
history = model.fit(train_images,
train_labels, epochs=10,
```

```
validation_data=(test_images,  
test_labels))
```

```
# 6. Evaluate Performance  
test_loss, test_acc =  
model.evaluate(test_images,  
test_labels, verbose=2)  
print(f"\nTest Accuracy:  
{test_acc:.4f}")
```

```
# Optional: Plot training history  
plt.plot(history.history['accuracy'],  
label='accuracy')  
plt.plot(history.history['val_accuracy'],  
label='val_accuracy')  
plt.xlabel('Epoch')  
plt.ylabel('Accuracy')  
plt.ylim([0, 1])  
plt.legend(loc='lower right')  
plt.show()
```

Output:



Conclusion:

This experiment successfully demonstrates the application of a Convolutional Neural Network (CNN) for multi-class image classification using the CIFAR-10 dataset. By leveraging specialized layers—specifically convolutional filters and max-pooling—the model effectively extracted complex spatial features from color images.

While the baseline model establishes a solid predictive foundation, the experiment highlights that raw architecture alone is rarely sufficient for optimal performance. The hyperparameter tuning phase proved essential; adjustments to learning rates, batch sizes, and the introduction of regularization techniques like dropout were critical for mitigating overfitting and improving the model's generalization to unseen test data. Ultimately, this workflow illustrates both the powerful pattern-recognition capabilities of CNNs and their inherent requirements for careful optimization and significant computational resources.

